

CSE 0613-2203 | Design and Analysis of Algorithms

Topic: Introduction of Algorithms

1. Algorithm

An **algorithm** is a step-by-step procedure that defines a set of instructions to be executed in a certain order to get the desired output.

| Key Points

- Step-by-step procedure
- Produces desired output
- Language independent
- Can be implemented in different programming languages

2. Categories of Algorithms

- **Search:** Finding an item in a data structure.
- **Sort:** Arranging items in a particular order.
- **Insert:** Adding an item to a data structure.
- **Update:** Modifying an existing item.
- **Delete:** Removing an item.

3. Origin of the Word Algorithm

The word **Algorithm** comes from the name of the Persian mathematician:

Abu Ja'afar Mohammed Ibn Mousa Al Khwarizmi (825 A.D.)

He wrote several books on Mathematics and Algebra.

4. Characteristics of an Algorithm

| 1. Unambiguous

Every step must be **clear and have only one meaning**.

| 2. Input

An algorithm can have **zero or more well-defined inputs**.

| 3. Output

An algorithm must have **one or more well-defined outputs**.

| 4. Finiteness

An algorithm must **terminate after a finite number of steps**.

| 5. Feasibility

The algorithm must be possible with the available resources.

6. Independent

An algorithm should be independent of programming code/language.

Remember

U → **I** → **O** → **F** → **F** → **I**

5. Ways to Express an Algorithm

Natural Language

Written using plain English.

- Easy to describe
- May become unclear for complex problems

Flowchart

A graphical representation of algorithm steps.

- Easy to visualize

Pseudocode

Text-based, code-like instructions without specific programming-language syntax.

- Easy to understand
- Considered a good way to express an algorithm

6. Steps to Design an Algorithm

1. **Problem Definition** – Clearly define the problem.
2. **Constraints** – Identify limitations or rules.
3. **Inputs** – Determine the given data.
4. **Outputs** – Specify the expected result.
5. **Solution Feasibility** – Ensure the solution works within the constraints.

Shortcut

Problem → **Constraints** → **Input** → **Output** → **Feasibility**

7. Algorithm Analysis

A Priori Analysis

A theoretical analysis performed before implementation.

- Mathematical analysis
- Assumes other factors are constant
- Uses **Ω** , **O** , and **θ**

A Posteriori Analysis

An empirical analysis performed after implementation.

- Algorithm is implemented and executed

- Actual running time and space are measured
- Example: **0.18 seconds, 8 MB**

Easy Difference

A Priori	A Posteriori
Before implementation	After implementation
Theoretical	Empirical
Mathematical counting	Run and measure
Ω , O , θ	Actual time and space

8. Algorithm Complexity

The two main factors that determine algorithm efficiency are:

Time Factor

Measures the time/number of key operations required by an algorithm.

Example: Number of comparisons in a sorting algorithm.

Space Factor

Measures the maximum memory space required by an algorithm.

Complexity Function

If n is the size of input data, the complexity $f(n)$ represents the running time and/or storage space required by the algorithm in terms of n .

Topic

Writing of Algorithm

1. Ways to Express an Algorithm

An algorithm can be expressed in three main ways:

1.1 Natural Language

Natural Language uses plain English (or another human language) to describe each step in a simple and logical order.

Advantages

- Easy to read and understand.
- Suitable for beginners.
- Suitable for small problems.

Disadvantage

- May become ambiguous for complex problems.
- Different people may interpret instructions differently.

1.2 Flowchart

A **flowchart** is a graphical representation of an algorithm or process.

It uses symbols/shapes such as:

- Arrows
- Rectangles
- Diamonds

These symbols explain the sequence of steps in an algorithm or process.

Uses

- Software development
- Business process modeling
- Engineering

Why Use Flowcharts?

- Provides clarity and simplifies complex processes.
- Makes algorithms easier to understand.
- Provides a universal visual language.
- Helps reduce miscommunication.
- Useful for documenting procedures and workflows.
- Makes training easier.
- Improves visualization of the problem.

2. Flowchart Symbols

Symbol	Name	Purpose
Oval	Terminal / Terminator	Represents Start, Stop, or Halt
Parallelogram	Input / Output	Represents input and output operations
Rectangle	Action / Process	Represents arithmetic instructions or operations
Diamond	Decision	Represents Yes/No or True/False decisions
Arrow	Flow Line	Shows the sequence and direction of execution
Small Circle	On-Page Connector	Connects different parts of a flowchart on the same page
Pentagon	Off-Page Connector	Connects flowchart parts located on different pages

Important Details

Terminal / Terminator:

The first and last symbol in a flowchart. It represents **Start, Stop, or Halt**.

Input / Output:

A parallelogram represents operations that take input or produce output.

Action / Process:

A rectangle represents an action or operation such as addition, subtraction, multiplication, or division.

Decision:

A diamond represents a decision point, such as a **Yes/No** or **True/False** condition.

Flow Lines:

Arrows indicate the exact sequence in which instructions are executed.

On-Page Connector:

Used when a flowchart is complex and the continuation remains on the **same page**.

Off-Page Connector:

Used when the continuation of a flowchart is on a **different page**.

3. Pseudocode

Pseudocode is a text-based, code-like way of describing an algorithm without using the syntax of a specific programming language.

It is:

- Easy to understand.
- Implementation-independent.
- Suitable for expressing algorithmic logic.
- Understandable even to beginners with basic knowledge.

Pseudocode is considered a **good way to express an algorithm**.

4. Core Pseudocode Constructs

Programming logic mainly flows through:

1. **Sequence**
2. **Decision**
3. **Loops / Iteration**
4. **Function Handling**
5. **Exception Management**

4.1 Sequential Operations

Sequential operations execute instructions **one after another**.

```
INPUT user_data
VALIDATE user_data
PROCESS user_data
OUTPUT results
```

The instructions are executed in the order in which they are written.

5. Decision Making

Decision making uses conditional statements to create branching logic.

IF-THEN-ELSE

```
IF age < 18 THEN
    DISPLAY "Access denied"
ELSE
    GRANT access
ENDIF
```

Important Point

- **IF** statements are suitable for Boolean conditions and complex logical expressions.
- **CASE** statements are useful when multiple values of a single variable need to be evaluated.

5.1 CASE Statement

```
CASE payment_method OF
    "credit": PROCESS credit_payment
    "debit": PROCESS debit_payment
    "crypto": PROCESS crypto_payment
ENDCASE
```

IF vs CASE:

IF	CASE
Suitable for Boolean conditions	Suitable for multiple values of one variable
Good for complex logical expressions	Clarifies multiple conditional scenarios

6. Loops and Iterations

Loops execute a block of instructions repeatedly based on specified conditions.

They are useful for:

- Processing collections of data.
- Performing repeated calculations.
- Maintaining continuous operations.

6.1 FOR Loop

Use a **FOR loop** when the exact number of iterations is known.

```
FOR each item IN shopping_cart
  CALCULATE item_total
  ADD TO cart_total
ENDFOR
```

6.2 WHILE Loop

Use a **WHILE loop** when repetition depends on a dynamic condition.

```
WHILE connection_active
  PROCESS incoming_data
  CHECK connection_status
ENDWHILE
```

6.3 REPEAT-UNTIL Loop

A **REPEAT-UNTIL** loop checks the condition **after** executing the code block.

Therefore, it guarantees **at least one execution**.

```
REPEAT
  GET user_input
  VALIDATE input
UNTIL input_valid
```

Loop Comparison

Loop	Main Use
FOR	Exact number of iterations is known
WHILE	Repetition depends on a condition
REPEAT-UNTIL	Condition is checked after execution; runs at least once

7. Function Handling

Functions divide complex operations into smaller, manageable parts.

A function can:

- Accept parameters.
- Process data.
- Return results.
- Maintain scope isolation.

Example

```
FUNCTION validate_user(username, password)
    CHECK username_exists
    VERIFY password_hash
    RETURN validation_status
ENDFUNCTION
```

8. Exception Management

Runtime errors can occur when:

- Processing user input.
- Accessing external resources.
- Performing complex calculations.

TRY-CATCH blocks can be used to plan for possible errors.

Example

```
TRY
    PROCESS sensitive_data
CATCH data_error
    LOG error_details
    NOTIFY admin
ENDTRY
```

9. Guidelines for Writing Pseudocode

Good pseudocode should balance **expressiveness and readability** while clearly communicating the intended logic.

9.1 Syntax Guidelines

- Write keywords in **UPPERCASE**.
- Put each statement on its **own line**.
- Use clear keywords such as:
 - WHILE
 - IF
 - FUNCTION

Example:


```
IF condition THEN
    PROCESS data
ENDIF
```

9.2 Naming Conventions

Use **descriptive variable names** instead of cryptic names.

Good examples:

- `customer_record`
- `temp_buffer`

For functions, use **action verbs**:

- `calculate_total()`
- `validate_input()`
- `process_transaction()`

9.3 Structure and Flow

- Keep statements **concise but complete**.
- Pseudocode should read like a clear technical document.
- Maintain language independence.

Instead of:

```
x = x + 1
```

Prefer:

```
INCREMENT counter BY 1
```

9.4 Proper Indentation

Use indentation to show the logical hierarchy of the algorithm.

Example:

```
IF user_authenticated THEN
  FOR each_permission IN user_permissions
    IF permission_active THEN
      GRANT access_level
    ENDIF
  ENDFOR
ENDIF
```

Close Constructs Explicitly

Use:

- `ENDIF`
- `ENDWHILE`
- `ENDFOR`
- `ENDCASE`
- `ENDFUNCTION`
- `ENDTRY`

These markers make the pseudocode easier to understand and prevent logical confusion.

10. Example: Linear Search in Natural Language

Problem

Search for a value **X** in an array **A** containing **n** elements.

Algorithm

1. Start.
2. Read the array **A**, number of elements **n**, and search value **X**.
3. Set index **i = 0**.
4. Compare **A[i]** with **X**.
5. If **A[i] = X**, display "**Element found at index i**" and stop.
6. Otherwise, move to the next element: **i = i + 1**.
7. Repeat the comparison until all elements have been checked.
8. If the end of the array is reached and **X** is not found, display "**Element not found.**"
9. Stop.

Example

```
Array: [10, 25, 30, 45, 60]
Search Key: 45
```

11. Example: Largest of Three Numbers

Problem

Find the largest of three numbers.

Pseudocode

```
START

1. Read three numbers: num1, num2, num3

2. IF num1 > num2 AND num1 > num3 THEN
    largest = num1
ELSE IF num2 > num3 THEN
    largest = num2
ELSE
    largest = num3
ENDIF

3. Print largest

END
```

Quick Exam Revision

Most Important Topics

1. Ways to express an algorithm

- Natural Language
- Flowchart
- Pseudocode

2. Flowchart symbols

- Terminator
- Input/Output
- Process
- Decision
- Flow line
- On-page connector
- Off-page connector

3. Pseudocode constructs

- Sequence
- IF-THEN-ELSE
- CASE

- FOR
- WHILE
- REPEAT-UNTIL
- Functions
- TRY-CATCH

4. Pseudocode writing guidelines

- Uppercase keywords
- One statement per line
- Descriptive names
- Proper indentation
- Explicit END statements

5. Important examples

- Linear Search
- Largest of Three Numbers

One-Line Memory Guide

Algorithm Expression → Natural Language → Flowchart → Pseudocode

Flowchart → Start/Stop → Input/Output → Process → Decision → Flow → Connector

Pseudocode → Sequence → Decision → Loop → Function → Exception

Loops → FOR (known iterations) → WHILE (condition-based) → REPEAT-UNTIL (runs at least once)